

Sieci neuronowe stworzone algorytmem Neat dla drona latającego w środowisku niestacjonarnym

Sprawozdanie projektu zaliczeniowego na przedmiot Modelowanie i Symulacja Systemów

Radosław Mocarski
Tomasz Zapart

April 2025

Wprowadzenie

Celem projektu jest stworzenie uproszczonego symulatora drona, który ma za zadanie opryskać elewację budynku. Zadanie to jest utrudnione niestacjonarnością środowiska w postaci otwierających się okien budynku i zmieniającego się wiatru wpływającego na drona. Dron ma za zadanie opryskać całą elewację w możliwie najkrótszym czasie z jak największą dokładnością, bez żadnej kolizji.

Zastosowane technologie

Do przeprowadzenia symulacji został zastosowany silnik **Unity**, do którego kod był pisany w języku **C#**. Wybór ten został dokonany z następujących powodów:

- Znajomość tego silnika przez twórców projektu,
- Uniwersalność narzędzia,
- Bogato zaimplementowana fizyka,
- Liczne publicznie dostępne modele obiektów tworzone specjalnie pod ten silnik.

Silnik Unity jest popularnym narzędziem także do symulacji pracy dronów, co dodatkowo uzasadnia jego wybór.^{1 2}

¹<https://card-file.ontu.edu.ua/items/89dd4dc5-db57-4499-bca7-1fee21301486>

²<https://yadda.icm.edu.pl/baztech/element/bwmeta1.element.baztech-07aeb349-4fc3-40b6-9b09-c352e1915a22>

Algorytm NEAT

Algorytm **NEAT** (Neuroevolution of Augmenting Topologies) jest algorytmem, który nie posiada z góry określonej topologii sieci neuronowej. Dodatkowo, algorytm ten jednocześnie nadbudowuje sieć neuronową, co ogranicza liczbę zbędnych neuronów. Skutkuje to przyspieszeniem późniejszego wykorzystania gotowej sieci, dając bardzo zbliżone, a czasem nawet lepsze rezultaty w porównaniu do tradycyjnych metod.

Przykładem zastosowania algorytmu NEAT są badania przeprowadzone przez badaczy z Kanadyjskiego Królewskiego Kolegium Wojskowego, którzy wykorzystali go do rozwiązania problemu *dynamicznego soaringu* dla małych, bezzałogowych statków powietrznych. W artykule przedstawiono matematyczny model lotu przy dynamicznym soaringu, uwzględniając różne warianty, takie jak:

- Liniowe,
- Wykładnicze,
- Logarytmiczne.

Porównano wyniki uzyskane za pomocą algorytmu NEAT z tradycyjnymi podejściami sterowania (np. metodami kolokacji). Zauważono, że tradycyjne metody, choć skuteczne, często prowadzi do rozwiązań lokalnych, silnie zależnych od początkowych warunków i mniej odpornych na zmiany środowiskowe. Algorytm NEAT umożliwił stworzenie sterownika o uproszczonej strukturze, który:

- Wymaga minimalnej liczby danych treningowych,
- Jest wystarczająco prosty, by móc go zaimplementować w czasie rzeczywistym na standardowych autopilotach,
- Wykazuje dużą odporność na zmienne warunki wiatru.

Testy porównawcze wykazały, że trajektorie generowane przez NEAT są porównywalne lub lepsze od tradycyjnych pod względem wydajności, przy jednoczesnym uproszczeniu sterowania.³ Choć problem soaringu różni się od naszego zadania, uniwersalność algorytmu NEAT czyni go wartym sprawdzenia w kontekście opryskiwania elewacji.

Biblioteka SharpNeat

Do projektu dodano bibliotekę **SharpNeat**, aby wykorzystać algorytm NEAT. Niestety, silnik Unity nie jest kompatybilny z najnowszymi wersjami .NET, co zmusiło nas do użycia starszej wersji tej biblioteki, niewspieranej od 2020 roku. Biblioteka ta, choć niszowa, była wcześniej wykorzystywana w podobnych projektach, np. przez badaczy z Uniwersytetu w Siedlcach, którzy stosowali bibliotekę *UnitySharpNeat* (oparta na *UnityNeat* i *SharpNeat*).⁴

³<https://arc.aiaa.org/doi/epdf/10.2514/6.2020-1946>

⁴<https://yadda.icm.edu.pl/baztech/element/bwmeta1.element.baztech-07aeb349-4fc3-40b6-9b09-c352e1915a22>

Dane wejściowe

Kluczowa rola w symulacji sterowania dronem odgrywają odpowiednio zdefiniowane dane wejściowe. W artykule *"Design and Simulation of a Neuroevolutionary Controller for a Quadcopter Drone"* autorzy uwzględnili następujące parametry w warstwie wejściowej modelu:

- Współrzędne środka masy – określające pozycję drona w przestrzeni w układzie odniesienia Ziemi,
- Predkość liniowa,
- Przyspieszenie liniowe,
- Kąty nachylenia – opisujące orientację drona,
- Predkości katowe – pochodna kątów orientacji względem czasu.

Parametry te zostały wybrane w celu zastosowania neuroewolucji do sterowania dronem, a głównym celem było stworzenie autonomicznego systemu zdolnego dotrzeć do określonej pozycji w przestrzeni. Do sterowania wykorzystano sieć neuronową typu **MLP** (Multilayer Perceptron), a proces uczenia oparto na algorytmie ewolucyjnym. Autorzy uzasadnili ten wybór trudnościami w zdefiniowaniu optymalnej trajektorii lotu – szczególnie w ustaleniu odpowiednich predkości wirników w odpowiedzi na zmieniające się stany drona. Funkcja kosztu w tym problemie była nieróżniczkowalna, co uniemożliwiło zastosowanie standardowych metod gradientowych.⁵

Funkcja aktywacji

W sieciach neuronowych kluczową rolę odgrywa funkcja aktywacji, która powinna być nieliniowa, aby lepiej modelować złożone zależności w danych. Problem ten omówiono w pracy *"Application of Artificial Neural Networks and NEAT Algorithm to Control a Quadcopter"*, gdzie autorzy zoptymalizowali manewr zwisu drona, istotny w praktycznych zastosowaniach. Porównano trzy funkcje aktywacji:

1. **Steepened Sigmoid:** $f(x) = \frac{1}{1+e^{-4.9x}}$,
2. **Sigmoid:** $f(x) = \frac{1}{1+e^{-x}}$,
3. **Gaussian:** $f(x) = e^{-x^2}$.

Najlepiej wypadła zmodyfikowana wersja sigmoidy (*Steepened Sigmoid*), natomiast klasyczna sigmoid, natomiast funkcja Gaussa zupełnie nie sprawdziła się w tym problemie.⁶

⁵<https://www.mdpi.com/2226-4310/10/5/418>

⁶<https://yadda.icm.edu.pl/baztech/element/bwmeta1.element.baztech-07aeb349-4fc3-40b6-9b09-c352e1915a22>